

# Creating a registry package

Spec-first, data-first. No runtime in the registry.

agents-repo/registry

# Registry role

This repository is the **source of truth** for:

- Normative specs/
- Package source under packages/
- Version ZIPs and manifests

It does **not** run agents. Runtime belongs in install targets and consumers (webapp, CLI).

# Package layout

```
packages/<namespace>/<package-id>/  
  metadata.json  
  README.md           # optional  
  agents/*.agent.md  
  flows/*.agent.md  
  versions/           # script output only
```

IDs: lowercase kebab-case `^[a-z0-9]+(?:-[a-z0-9]+)*$` .

# What you write vs scripts

**You write** (package root only):

- `metadata.json`, optional `README.md`
- `agents/` and `flows/` source + sidecar metadata

**Scripts write** (do not hand-edit):

- `versions/**`, `detail.json`, catalog `packages/index.json`

# Agent and flow files

- Extension: `.agent.md` (GitHub Copilot requirement)
- Frontmatter `name` MUST equal the file stem
- Matching `*.metadata.json` sidecars
- Agent IDs and flow IDs unique across the package

See `specs/agent-format.md` and `specs/flow-format.md`.

## Local: validate

```
npm run package:validate -- --package <namespace>/<package-id>
```

Checks source layout and metadata against specs **before** you generate artifacts.

# Local: build

```
npm run package:build -- --package <namespace>/<package-id>
```

Generates per-target ZIPs, source archive, checksums, `manifest.json`, `detail.json`, and updates `packages/index.json`.

ZIP names have **no** `v` prefix: `1.0.0.zip` style version folders.

# Local: validate artifacts

```
npm run package:validate-artifacts -- \
  --package <namespace>/<package-id> --version <version>
```

Structural and security checks on generated ZIPs. Required before you call the work done.

# GitHub workflow

1. Issue (package submission form, or omit per exception)
2. Branch `package/<slug>` or `package/<n>-<slug>`
3. **Draft** PR to upstream `main` (fork for external contributors)
4. Human marks ready; maintainer squash-merges

# Squash-merge titles

MUST use:

- `feat(package):` or `fix(package):`

Optional `!` emphasizes breaking **package** content in notes only. All package titles publish a registry **PATCH** so `v2.x` consumers update.

# Forks

External contributors SHOULD fork **agents-repo/registry**, work on the fork, and open a draft PR from `FORK_USER:branch` to upstream `main` .

Suggested authoring path after the draft PR: in-tree  
`full-package-creation-flow` .

# Specs to keep open

- `specs/package-format.md`
- `specs/metadata-schema.md`
- `specs/manifest-schema.md`
- `specs/agent-format.md` / `specs/flow-format.md`
- `specs/versioning-rules.md`

Do not invent rules that contradict these files.

# Templates

- `.github/ISSUE_TEMPLATE/package-submission.yml`
- `.github/ISSUE_TEMPLATE/package-correction.yml`
- `.github/CONTRIBUTING.md`
- Site guide: [Submit a package](#)

# Context

Org ecosystem PDF (publish vs browse vs install):

[ecosystem-overview.pdf](#)

This deck is **package authoring** only. CLI install is the cli deck. Proxy internals are the registry-proxy deck.

# Next

Copy an existing package under `packages/`, then `validate` → `build` → `validate-artifacts` → draft PR.